OPERATING SYSTEM

UNIT 5 I/O MANAGEMENT

Prof. N. K. PATIL
Department of Computer Engineering
Sinhgad Institute of Technology, Lonavala

Email nkp.sit@sinhgad.edu
Contact No.+91-9423340927

sppu_te_be_comp

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

SPPU Engineering & Technical UPDATES

WhatsApp channel

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

@SPPU_ENGINEERING_UPDATE

Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

sppu_te_be_comp

CONTENTS

1. System architecture, A canonical device, The canonical protocol
2. CPU Virtualization, Lowering CPU Overhead with Interrupts
3. More efficient data movement with DMA, Methods of device interaction
4. Fitting into the OS: The device driver, Case Study: A simple IDE disk driver
5. Hard disk drives, files and directories, The fast file system, file system implementation
6. Disk failure modes, Handling latent sector error
7. Detecting corruption: the checksum, using checksums
8. I/O Management in Linux/Windows Operating system

sppu_te_be_comp

COURSE OUTCOME

- CO5: **Elaborate** I/O management concepts.

sppu_te_be_comp

INTRODUCTION TO INPUT/OUTPUT

- I/O is quite critical to computer systems, of course; imagine a program without any input (it produces the same result each time); now imagine a pro gram with no output (what was the purpose of it running?).
- For computer systems to be interesting, both input and output are required.

sppu_te_be_comp

CATEGORIES OF I/O DEVICES

- **Human readable**
  - Used to communicate with the user
  - Printers
  - Video display terminals
    - Display
    - Keyboard
    - Mouse

sppu_te_be_comp

CATEGORIES OF I/O DEVICES

- **Machine readable**
  - Used to communicate with electronic equipment
  - Disk and tap drives
  - Sensors
  - Controllers
  - Actuators

sppu_te_be_comp

CATEGORIES OF I/O DEVICES

- **Communication**
  - Used to communicate with remote devices
  - Digital line drivers
  - Modems

sppu_te_be_comp

DIFFERENCES IN I/O DEVICES

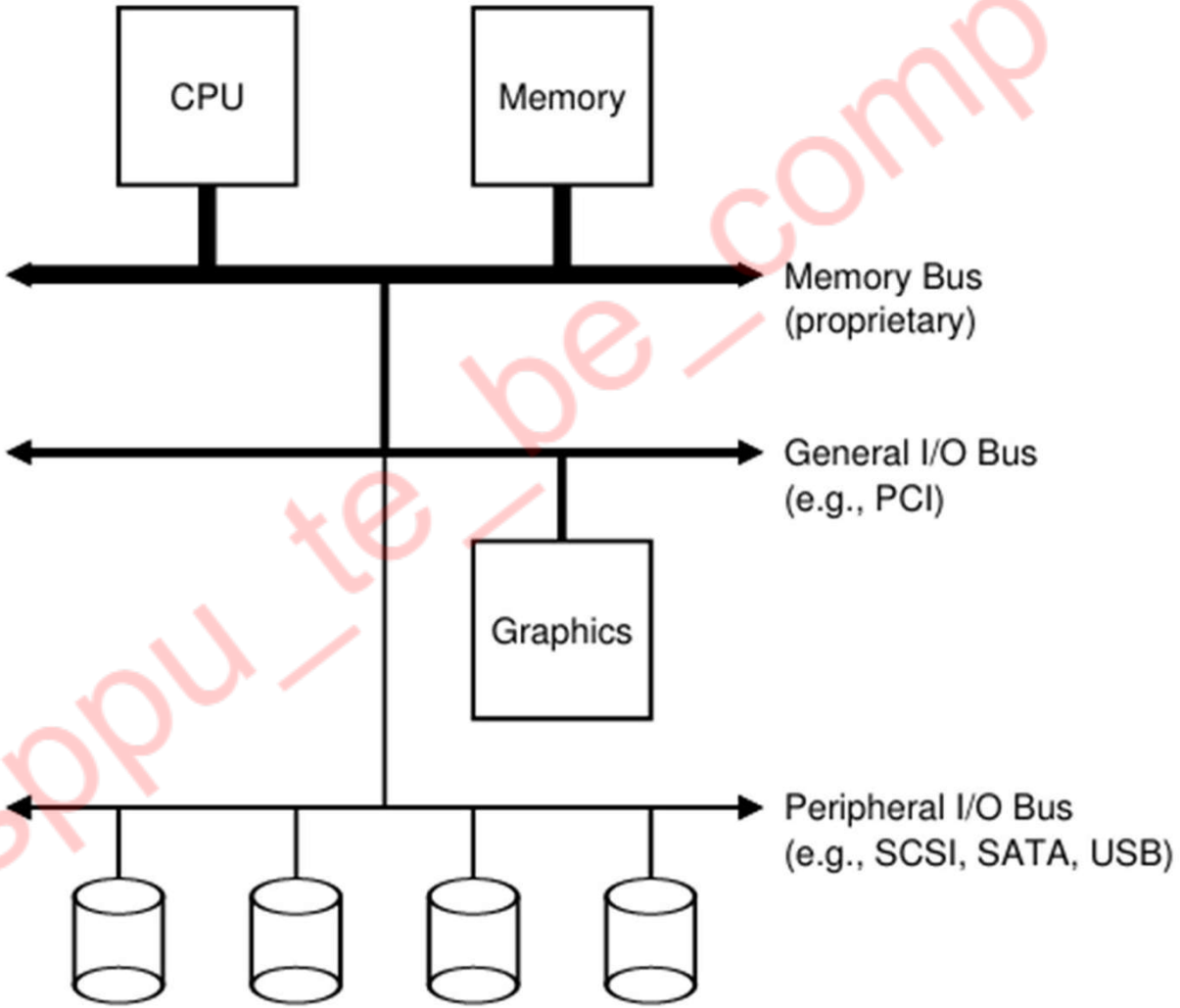
- **Data rate**
  - May be differences of several orders of magnitude between the data transfer rates
- **Application**
  - Disk used to store files requires file-management software
  - Disk used to store virtual memory pages needs special hardware and software to support it
  - Terminal used by system administrator may have a higher priority

sppu_te_be_comp

- **Complexity of control**
- **Unit of transfer**
  - Data may be transferred as a stream of bytes for a terminal or in larger blocks for a disk
- **Data representation**
  - Encoding schemes
- **Error conditions**
  - Devices respond to errors differently

sppu_te_be_comp

SYSTEM ARCHITECTURE



sppu_te_be_comp

SYSTEM ARCHITECTURE

- The picture shows a single CPU attached to the main memory of the system via some kind of **memory bus** or interconnect.
- Some devices are connected to the system via a general **I/O bus,** which in many modern systems would be **PCI** (or one if its many derivatives); graphics and some other higher-performance I/O devices might be found here.
- Finally, even lower down are one or more of what we call a **peripheral bus**, such as **SCSI, SATA, or USB**. These connect the slowest devices to the system, including **disks, mice**, and other similar components.

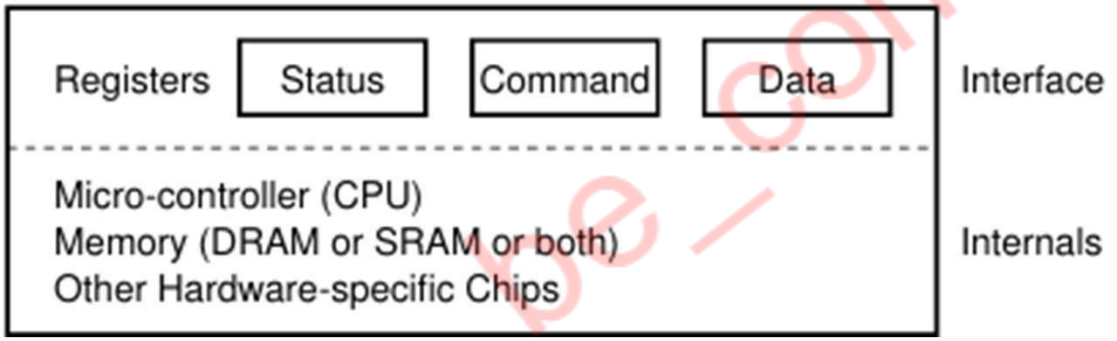
sppu_te_be_comp

- **Why do we need a hierarchical structure like this?**
  - System designers have adopted this hierarchical approach, where components that demands high performance (such as the graphics card) are nearer the CPU.
  - Lower performance components are further away.
  - The benefits of placing disks and other slow devices on a peripheral bus are manifold; in particular, you can place a large number of devices on it.

spu_te_be_comp

A CANONICAL DEVICE

- **Consider the below canonical device (not real).**



- Two important components are there.
  1. **Hardware interface** it presents to the rest of the system and protocol for typical interaction.
  2. **Internal structure.** This part of the device is implementation specific and is responsible for implementing the abstraction the device presents to the system.

spbu_to_be_comp

TECHNIQUES FOR PERFORMING I/O

- **Programmed I/O**
  - Process is busy-waiting for the operation to complete
- **Interrupt-driven I/O**
  - I/O command is issued. Processor continues executing instructions. I/O module sends an interrupt when done
- **Direct Memory Access (DMA)**
  - DMA module controls exchange of data between main memory and the I/O device
  - Processor interrupted only after entire block has been transferred

spu_to_be_comp

THE CANONICAL PROTOCOL

- In the previous diagram, the (simplified) device interface is comprised of three registers:
  - a **status** register, which can be read to see the current status of the device;
  - a **command** register, to tell the device to perform a certain task; and
  - a **data** register to pass data to the device, or get data from the device.
- By reading and writing these registers, the operating system can control device behavior.

spur_to_be_comp

- A typical interaction that the OS might have with the device in order to get the device to do something on its behalf. The protocol is as follows:

```
While (STATUS == BUSY)
    ; // wait until device is not busy
Write data to DATA register
Write command to COMMAND register
    (Doing so starts the device and executes the command)
While (STATUS == BUSY)
    ; // wait until device is done with your request
```

- **Note: -** one of the technique to interact with the device is **polling** (just asking it what is going on).

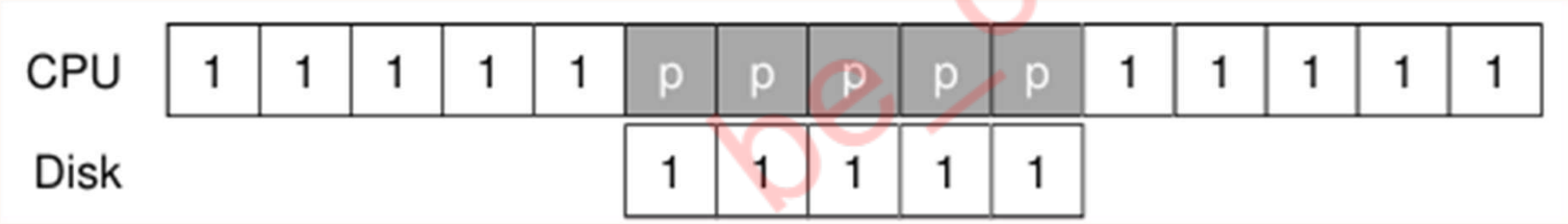
sppu_le_be_comp

LOWERING CPU OVERHEAD WITH INTERRUPTS

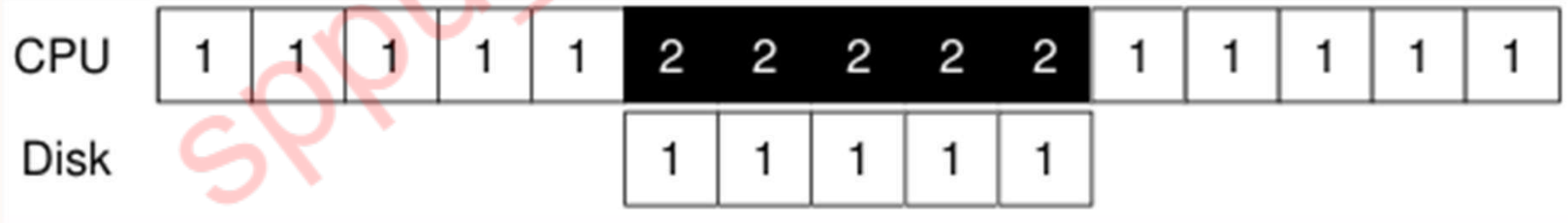
- Instead of **polling** the device repeatedly, the OS can issue a request, put the calling process to sleep, and context switch to another task.
- When the device is finally finished with the operation, it will raise a hardware interrupt, causing the CPU to jump into the OS at a pre-determined **interrupt service routine (ISR)** or more simply **an interrupt handler**.
- The handler is part OS code that will finish the request and wake the process waiting for the I/O.

spit_fe_be_comp

- Interrupts thus allow for overlap of computation and I/O, which is key for improved utilization. This timeline shows the problem:



- If instead we utilize interrupts and allow for overlap, the OS can do something else while waiting for the disk:



sppu_te_be_comp

MORE EFFICIENT DATA MOVEMENT WITH DMA

- **Problem with other technique:**



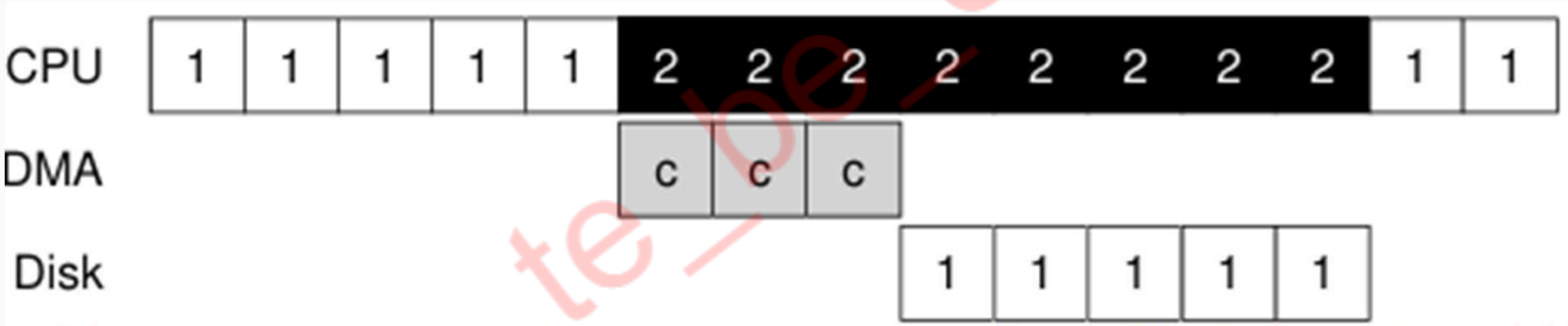
- In the timeline, Process1 is running and then wishes to write some data to the disk. It then initiates the I/O, which must copy the data from memory to the device explicitly, one word at a time (marked c in the diagram). When the copy is complete, the I/O begins on the disk and the CPU can finally be used for something else.

sppu_te_be_comp

- The solution to this problem is something we refer to as Direct Memory Access (DMA).
- A DMA engine is essentially a very specific device within a system that can orchestrate transfers between devices and main memory without much CPU intervention.
- DMA works as follows. To transfer data to the device, for example, the OS would program the DMA engine by telling it where the data lives in memory, how much data to copy, and which device to send it to.
- At that point, the OS is done with the transfer and can proceed with other work.

sppu_te_be_comp

- When the DMA is complete, the DMA controller raises an interrupt, and the OS thus knows the transfer is complete. The revised timeline:



- Here copying of data is now handled by the DMA controller. Because the CPU is free during that time, the OS can do something else, here choosing to runProcess2. Process2 thus gets to use more CPU before Process 1 runs again.

snpu_te_be_comp

METHODS OF DEVICE INTERACTION

- Two primary methods of device communication have developed.
- **Explicit I/O instructions:** These instructions specify a way for the OS to send data to specific device registers and thus allow the construction of the protocols described above.
- **Memory mapped I/O:** The hardware makes device registers available as if they were memory locations. To access a particular register, the OS issues a load (to read)or store (to write) the address; the hardware then routes the load/store to the device instead of main memory.

spu_te_be_comp

FITTING INTO THE OS: THE DEVICE DRIVER

- At the lowest level, a piece of software in the OS must know in detail how a device works known as device driver, and any specifics of device interaction are encapsulated within.
- A device driver is specialized software that acts as a translator between an operating system (OS) and a hardware device.
- By converting high-level OS instructions into low-level hardware commands, a driver enables seamless communication, ensuring that devices like printers, graphics cards, and mice function correctly.

sppu_te_be_comp

HARD DISK DRIVES

- The basic interface for all modern drives is straightforward. The drive consists of a large number of sectors (512-byte blocks), each of which can be read or written.
- The sectors are numbered from 0 to n − 1 on a disk with n sectors. Thus, we can view the disk as an array of sectors; 0 to n − 1 is thus the address space of the drive.
- Multi-sector operations are possible; indeed, many file systems will read or write 4KB at a time (or more).

sppu_te_be_comp

- **Few terms related to disk drives are:**
  - **Platter:** A circular hard surface on which data is stored persistently by inducing magnetic changes to it. These platters are usually made of some hard material (such as aluminum), and then coated with a thin magnetic layer that enables the drive to persistently store bits even when the drive is powered off.
  - **Surface:** A disk may have one or more platters; each platter has 2 sides, each of which is known as surface.
  - The platters are all bound together around the **spindle**, which is connected to a motor that spins the platters around at a constant (fixed) rate.

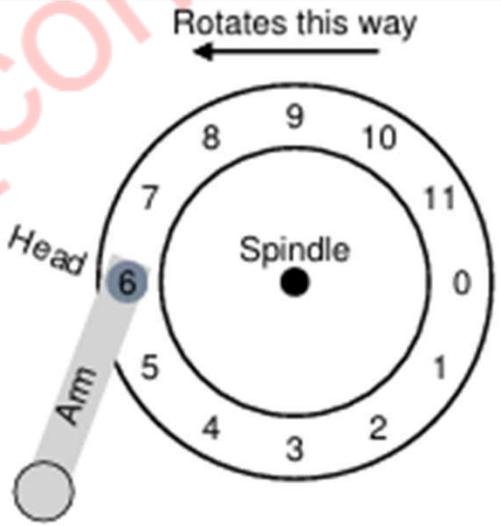
sppu_te_be_comp

- The rate of rotation is often measured in **rotations per minute (RPM).**
- Data is encoded on each surface in concentric circles of sectors; we call one such concentric circle a **track**.
- Process of reading and writing is accomplished by the **disk head**; there is one such head per surface of the drive.
- The disk head is attached to a single **disk arm**, which moves across the surface to position the head over the desired track.

sppu_te_be_comp

A SIMPLE DISK DRIVE

- This track has just 12 sectors, each of which is 512 bytes in size and addressed therefore by thenumbers0through 11.
- The single platter rotates around the spindle, to which a motor is attached.
- To read or write those sectors, and we need a disk head, attached to a disk arm

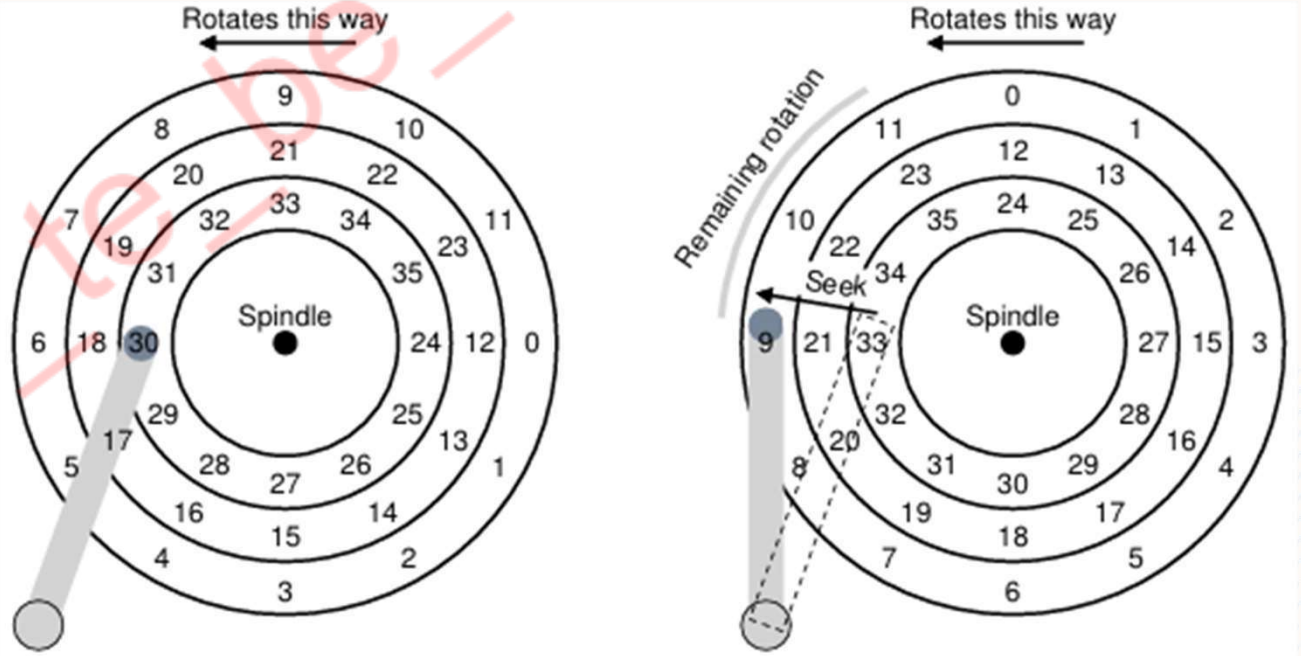


A Single Track Plus A Head

sppu_te_be_comp

The head is currently positioned over the innermost track (which contains sectors 24 through 35); the next track over contains the next set of sectors (12 through 23), and the outermost track contains the first sectors (0 through 11).

e.g., a read to sector 11.
To service this read, the drive has to first move the disk arm to the correct track (in this case, the outermost one), in a process known as a **seek**.
Seeks, along with rotations, are one of the most costly disk operations.



Three Tracks Plus A Head (Right: With Seek)

sppu_to_be_comp

- The seek, has many phases:
  1. **An acceleration** phase as the disk arm gets moving;
  2. Then **coasting** as the arm is moving at full speed, then **deceleration** as the arm slows down;
  3. Finally **settling** as the head is carefully positioned over the correct track.
- After the seek, the disk arm has positioned the head over the right track.
- A short rotational delay to complete the transfer.

spu_fe_be_comp

- When sector 11 passes under the disk head, the final phase of I/O will take place, known as the **transfer,** where data is either read from or written to the surface.
- And thus, we have a complete picture of I/O time: first a seek, then waiting for the rotational delay, and finally the transfer.
- I/O time is represented as the sum of three major components:

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

sppu_te_be_comp

DISK SCHEDULING

- Because of the high cost of I/O, the OS has historically played a role in deciding the order of I/Os issued to the disk.
- Given a set of I/O requests, the disk scheduler examines the requests and decides which one to schedule next.

sppu_te_be_comp

DISK SCHEDULING ALGORITHM

The reason for disk scheduling is to decrease the total seek time.

The different disk scheduling algorithms are:

- FCFS (First Come First Serve) Scheduling Algorithm
- SSTF (Shortest Seek Time First) Scheduling Algorithm
- SCAN Scheduling Algorithm
- C-SCAN Scheduling Algorithm
- LOOK Scheduling Algorithm
- C-LOOK Scheduling Algorithm

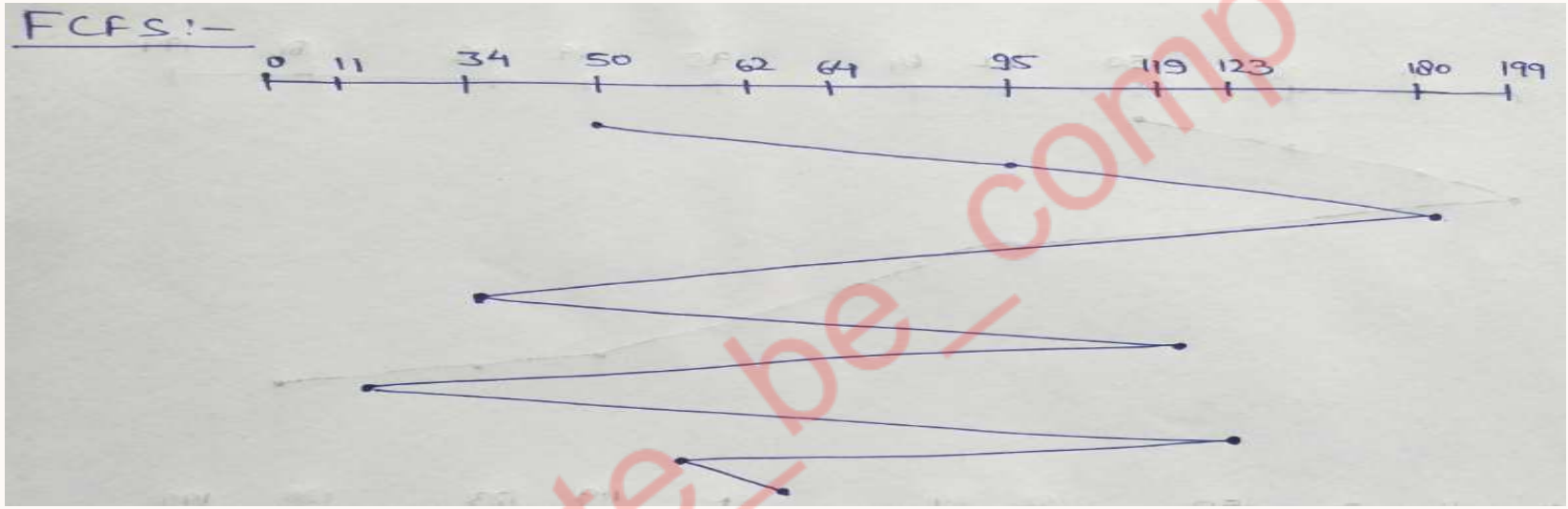
sppu-te-be-comp

DISK SCHEDULING ALGORITHM

- **First Come First Serve-** Least complex, the requests are accessed in the order in which they arrive in the disk queue.
- Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.

sppu_te_be_comp

DISK SCHEDULING ALGORITHM



Total Seek Time = |50 – 95| + |95 – 180| + |180 – 34| + |34 – 119| + |119 – 11| + |11 - 123| + |123-62| + |62 – 64|

= 45 + 85 + 146 + 85 + 108 + 112 + 61 + 2

= 644

Average Seek Time = 644/8 = 80.5

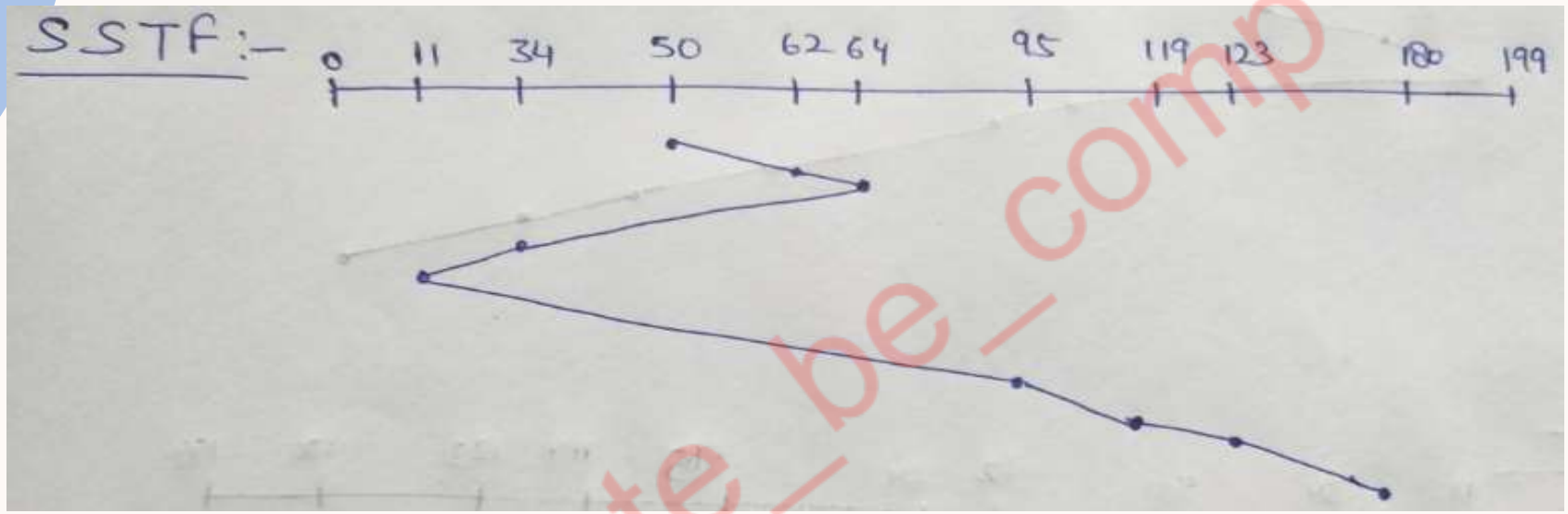
spu_te_be_comp

DISK SCHEDULING ALGORITHM

- **Shortest Seek Time First-** demands having most shortest seek time are executed first.
- Seek time of each request is determined ahead of time.
- Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.

sppu_te_be_comp

DISK SCHEDULING ALGORITHM



Total Seek Time = |50 – 62| + |62 – 64| + |64 - 34| + |34 – 11| + |11 – 95| + |95 - 119| + |119 - 123| + |123 – 180|

= 12 + 02 + 30 + 23 + 84 + 24 + 04 + 57

= 236

Average Seek Time = 236/8 = 29.5

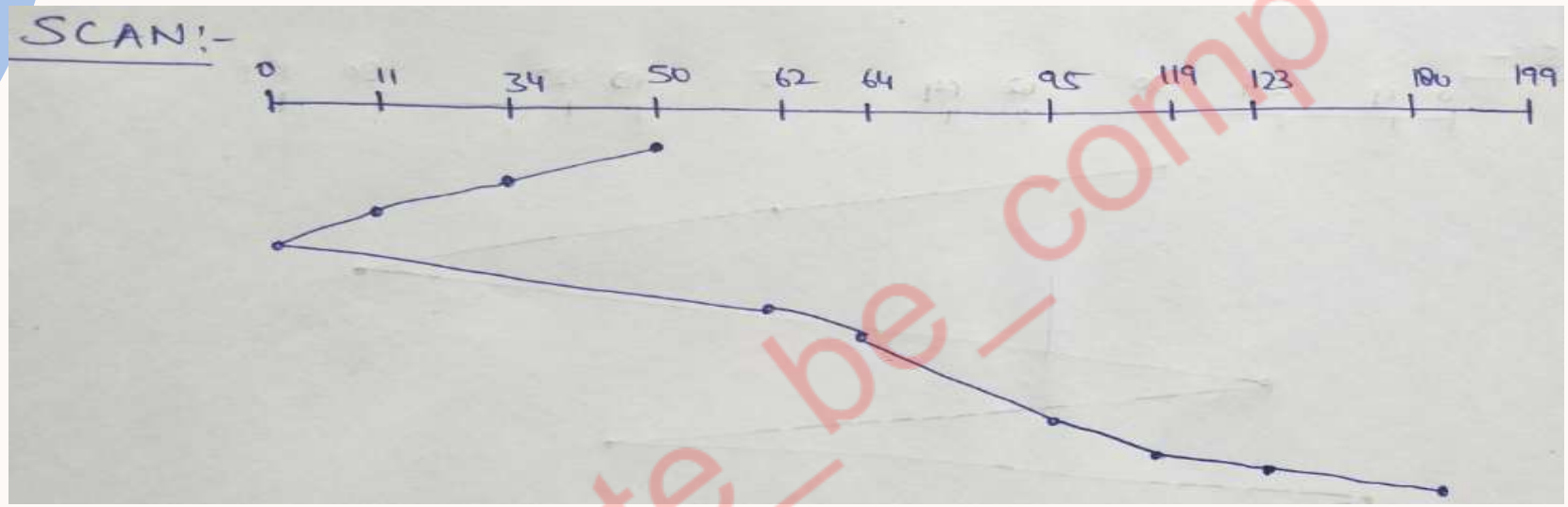
spu_te_be_comp

DISK SCHEDULING ALGORITHM

- **SCAN Scheduling-** also known as Elevator algorithm.
- The disk arm moves into a specific head till end fulfilling every request, & then it turns back & moves in opposite direction.
- Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.

sppu_te_be_comp

DISK SCHEDULING ALGORITHM



Total Seek Time = |50 – 34| + |34 – 11| + |11 - 0| + |0 – 62| + |62 – 64| + |64 - 95| + |95 - 119| + |119 – 123| + |123 – 180|

= 16 + 23 + 11 + 62 + 2 + 31 + 24 + 4 + 57

= 230

Average Seek Time = 230/8 = 28.75

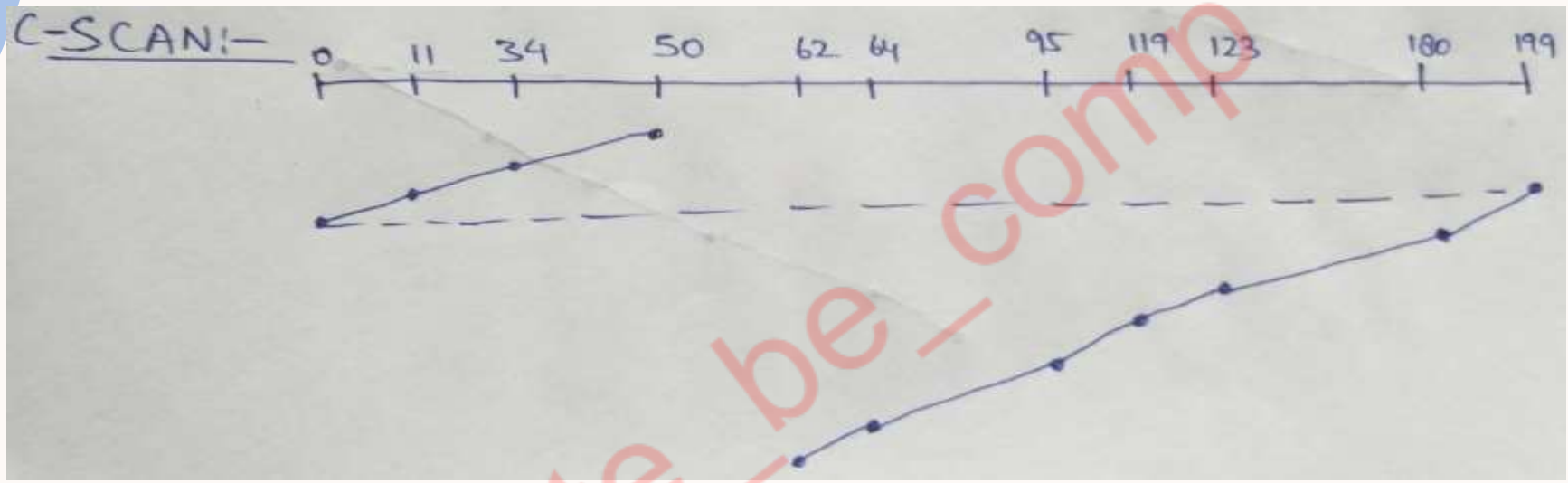
sppu_te_be_comp

DISK SCHEDULING ALGORITHM

- **C-SCAN Scheduling-** disk moves a specific way overhauling requests until it arrives at the last cylinder.
- Then it turns to the last disk of the other way without fulfilling any request then it turns back and start moving toward that path adjusting the rest of the requests.
- Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.

sppu_te_be_comp

DISK SCHEDULING ALGORITHM



Total Seek Time = |50 – 34| + |34 – 11| + |11 - 0| + |199 – 180| + |180 – 123| + |123 - 119| + |119 – 95| + |95 – 64| + |64 - 62|

= 16 + 23 + 11 + 19 + 57 + 4 + 24 + 31 + 2

= 187

Average Seek Time = 187/8 = 23.375

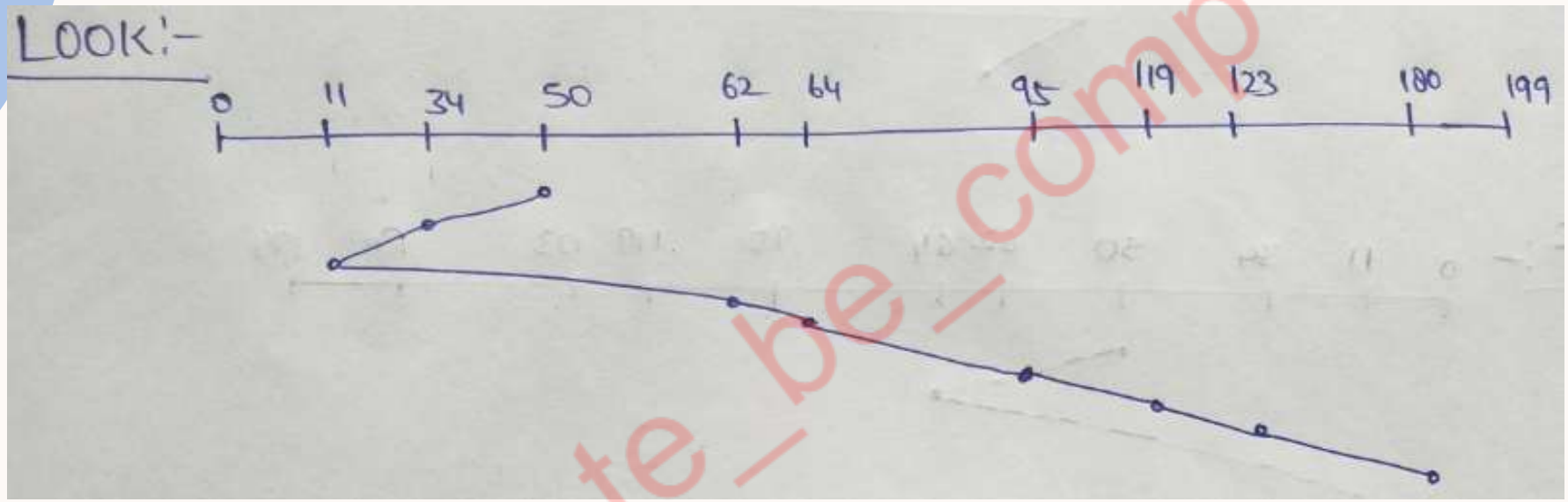
sppu_te_be_comp

DISK SCHEDULING ALGORITHM

- **LOOK Scheduling-** same as SCAN. Disk arm regardless of the end goes just to the last request.
- Then turns back and fulfill rest of the reqeust.
- Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.

sppu_te_be_comp

DISK SCHEDULING ALGORITHM



Total Seek Time = |50 – 34| + |34 – 11| + |11 - 62| + |62 – 64| + |64 - 95| + |95 - 119| + |119 – 123| + |123 – 180|

= 16 + 23 + 51 + 2 + 31 + 24 + 4 + 57

= 208

Average Seek Time = 208/8 = 26

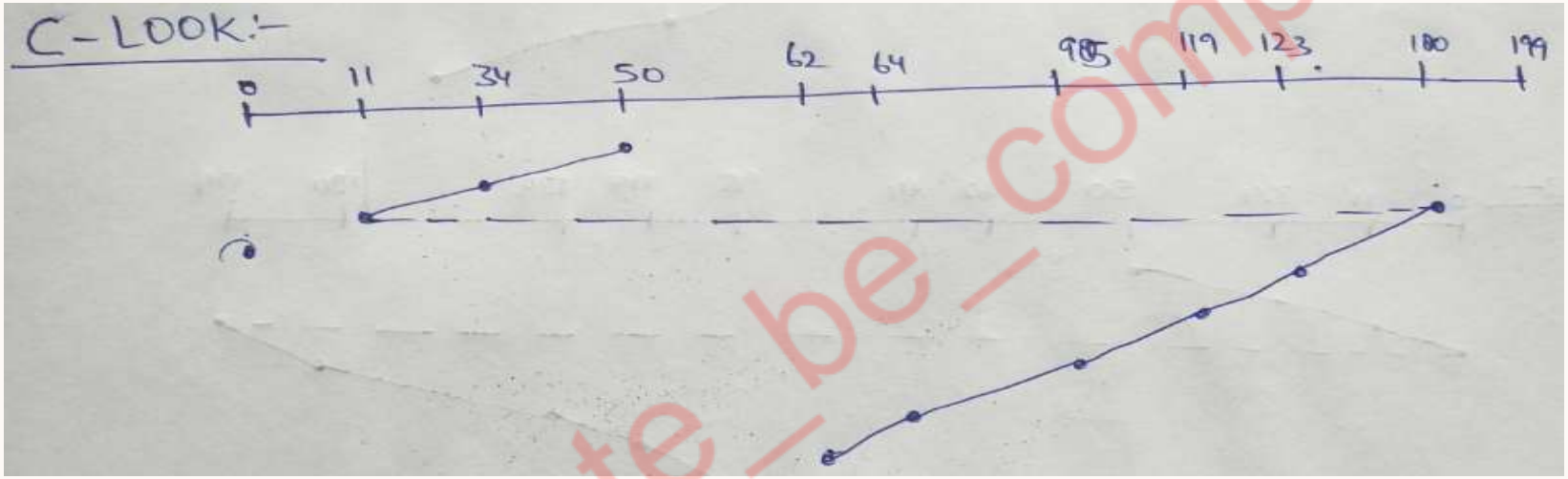
spcu_te_be_comp

DISK SCHEDULING ALGORITHM

- **C-LOOK Scheduling-** same as C-SACN. The disk arm goes to the last request and turn head and goes to the other end.
- Then turns back to fulfill remaining request.
- Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.

sppu_te_be_comp

DISK SCHEDULING ALGORITHM



Total Seek Time = |50 – 34| + |34 – 11| + |180 - 123| + |123 – 119| + |119 - 95| + |95 - 64| + |64 – 62|

= 16 + 23 + 57 + 4 + 24 + 31 + 2

= 157

Average Seek Time = 157/8 = 19.625

spu_te_be_comp

EXAMPLE

- **Suppose the order of request is- (82,170,43,140,24,16,150) and current position of Read/Write head is: 50.**
- **Calculating the total movement (seek time) of the disk's read/write head for a given set of requests by using algorithms like FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK.**

sppu_te_be_comp

FILES AND DIRECTORIES

- Two key abstractions have developed over time in the virtualization of storage.
  1. **File**
  2. **Directory**

- A **file** is simply a linear array of bytes, each of which you can read or write.
- Each entry in a **directory** refers to either files or other directories.

sppu_te_be_comp

FILE CONCEPT

- A file is a named collection of related information that is recorded on secondary storage.
- A file is a sequence of bits, bytes, lines, or records, the meaning of which is defined by the file's creator and user.
- Many different types of information may be stored in a file—source or executable programs, numeric or text data, photos, music, video, and so on.

sppu_te_be_comp

- **A file has a certain defined structure, which depends on its type.**
  - A **text file** is a sequence of characters organized into lines (and possibly pages).
  - A **source file** is a sequence of functions, each of which is further organized as declarations followed by executable statements.
  - An **executable file** is a series of code sections that the loader can bring into memory and execute.

sppu_te_be_comp

FILE ATTRIBUTES

Name – only information kept in human-readable form

Identifier – unique tag (number) identifies file within file system

Type – needed for systems that support different types

Location – pointer to file location on device

Size – current file size

Protection – controls who can do reading, writing, executing

Time, date, and user identification – data for protection, security, and usage monitoring

Information about files are kept in the directory structure, which is maintained on the disk

Many variations, including extended file attributes such as file checksum

Information kept in the directory structure

sppu_te_be_comp

FILE OPERATIONS

- **File is an abstract data type**
  - **Create**
  - **Write –** at write pointer location
  - **Read –** at read pointer location
  - **Reposition within file -** seek
  - **Delete**
  - **Truncate**
  - **Open(Fi) –** search the directory structure on disk for entry Fi, and move the content of entry to memory
  - **Close (Fi) –** move the content of entry Fi in memory to directory structure on disk

sppu_te_be_comp

DIRECTORY STRUCTURE

- It can be characterized as collection of the related files on the disk.
- Entry is kept for each file in the directory which stores all data identified with that file.
- Following is the data kept in a directory:
  - Name
  - Type
  - Location
  - Size
  - Position
  - Security
  - Use
  - Mounting

sppu_te_be_comp

DIRECTORY STRUCTURE CONTINUED…

Operations that are to be execute on a directory are:

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

sppu_te_be_comp

DISK FAILURE MODES

- Modern disks will occasionally seem to be mostlyworkingbuthavetroublesuccessfullyaccessingoneormo reblocks.
- Specifically, two types of single-block failures are common and worthy of consideration:
  - Latent-sector errors (LSEs) and
  - Block corruption.

spu_te_be_comp

1. LATENT SECTOR ERRORS (LSES)

- LSEs occur when a disk sector or group of sectors becomes damaged.
- These errors make the data in affected sectors unreadable.
- Causes of LSEs
  - Head Crash: Disk head touches the surface, damaging it.
  - Cosmic Rays: Can flip bits, corrupting data.

sppu_te_be_comp

- Drives use Error Correcting Codes (ECC) to:
  - Detect if bits in a block are valid.
  - Attempt to fix corrupted bits.

- If ECC cannot repair the damage:
  - The disk returns an error during read requests.
  - Data in the affected sector becomes inaccessible.

sppu_te_be_comp

- Some additional findings about LSEs are:
  - Costly drives with more than one LSE are as likely to develop additional errors as cheaper drives
  - For most drives, annual error rate increases in year two • LSEs increase with disk size
  - Most disks with LSEs have less than 50
  - Disks with LSEs are more likely to develop additional LSEs
  - There exists a significant amount of spatial and temporal locality
  - Disk scrubbing is useful (most LSEs were found this way)

sppu_te_be_comp

HANDLING LATENT SECTOR ERRORS

- Latent sector errors are rather straightforward to handle, as they are easily detected.
- When a storage system tries to access a block, and the disk returns an error, the storage system should simply use whatever redundancy mechanism it has to return the correct data.
- Easily detected problems such as LSEs are readily recovered through standard redundancy mechanisms.
- For e.g. In a mirrored RAID, the system should access the alternate copy.

spou_te_be_comp

- The growing prevalence of LSEs has influenced RAID designs over the years.
- When an entire disk fails, the RAID tries to reconstruct the disk (say, onto a hot spare) by reading through all of the other disks in the parity group and recomputing the missing values.
- If, during reconstruction, an LSE is encountered on any one of the other disks, we have a problem: the reconstruction cannot successfully complete.

spou_te_be_comp

DETECTING CORRUPTION: THE CHECKSUM

- Unlike latent sector errors, detection of corruption is a key problem.
- The primary mechanism used by modern storage systems to preserve data integrity is called the **checksum.**
  - A checksum is simply the result of a function that takes a chunk of data (say a 4KB block) as input,
  - then computes a function over said data, producing a small summary of the contents of the data (say 4 or 8 bytes).
  - This summary is referred to as the checksum.

spput_be_comp

- The goal of such a computation is to enable a system to detect:
  - if data has somehow been corrupted or altered by storing the checksum with the data and
  - then confirming upon later access that the data's cur rent checksum matches the original storage value.

- File systems use checksums to verify the integrity of stored data. This prevents file corruption due to disk errors or physical damage. Systems like ZFS employ checksums for built-in error correction.

spu_fe_be_comp

COMMON CHECKSUM FUNCTIONS

- One simple checksum function that some use is based on exclusive or (XOR).
  1. With XOR-based checksums, the checksum is computed sim ply by XOR'ing each chunk of the data block being checksummed, thus producing a single value that represents the XOR of the entire block.

sppu_te_be_comp

To make this more concrete, imagine we are computing a 4-byte checksum over a block of 16 bytes (this block is of course too small to really be a disk sector or block, but it will serve for the example). The 16 data bytes, in hex, look like this:

```
365e c4cd ba14 8a92 ecef 2c3a 40be f666
```

If we view them in binary, we get the following:

```
0011 0110 0101 1110    1100 0100 1100 1101
1011 1010 0001 0100    1000 1010 1001 0010
1110 1100 1110 1111    0010 1100 0011 1010
0100 0000 1011 1110    1111 0110 0110 0110
```

Because we've lined up the data in groups of 4 bytes per row, it is easy to see what the resulting checksum will be: simply perform an XOR over each column to get the final checksum value:

```
0010 0000 0001 1011    1001 0100 0000 0011
```

The result, in hex, is 0x201b9403.

spou_te_be_comp

2. Another simple checksum function is addition.

- This approach has the advantage of being fast; computing it just requires performing 2's complement addition over each chunk of the data, ignoring overflow.
- It can detect many changes in data, but is not good if the data, for example, is shifted.

sppu_te_be_comp

3. One final commonly-used checksum is known as a **cyclic redundancy check (CRC).**

- Assume you wish to compute the checksum over a data block D. All you do is treat D as if it is a large binary number and divide it by an agreed upon value (k).
- The remainder of this division is the value of the CRC.

sppu_te_be_comp

CHECKSUM LAYOUT

- The disk layout without checksums looks like this:



- The most basic approach simply stores a checksum with each disk sector (or block). Given a data block D, let us call the checksum over that data C(D).

sppu_te_be_comp

- With checksums, the layout adds a single checksum for every block:



- One problem is there that how to achieve this layout:
- Since, checksums are usually small (e.g., 8 bytes), and disks only can write in sector-sized chunks (512 bytes) or multiples.
- Solution: drive manufacturers format the drive with 520-byte sectors; an extra 8 bytes per sector can be used to store the checksum.

sppu_te_be_comp

- In disks that don't have such functionality, the file system must figure out a way to store the checksums packed into 512-byte blocks. One such possibility is as follows:



sppu_te_be_comp

USING CHECKSUMS

- How OS uses checksum?
- When reading a block D, the client (i.e., file system or storage controller) also reads its checksum from disk Cs(D), which we call the stored checksum.
- The client then computes the checksum over the retrieved block D, which we call the computed checksum Cc(D).

sppu_te_be_comp

- At this point, the client com pares the stored and computed checksums;
  - If they are equal (i.e., Cs(D) == Cc(D), the data has likely not been corrupted, and thus can be safely returned to the user.
  - If they do not match (i.e., Cs(D) != Cc(D)), this implies the data has changed since the time it was stored

sppu_te_be_comp

- Given a corruption, what should we do about it?
  - If the storage system has a redundant copy, the answer is easy: try to use it instead.
  - If the storage system has no such copy, the likely answer is to return an error.

sppu_te_be_comp

sppu_te_be_comp

sppu_te_be_comp

sppu_te_be_comp

sppu_te_be_comp

sppu_te_be_comp

THANK YOU

sppu_te_be_comp